



① What is a Request?

A Request is what the client sends to the server.

① Method → GET, POST, PUT, DELETE

② URL → / users / 5

③ Headers → Extra info

④ Body → Data (mostly in POST/PUT)

Analogy

Sending a parcel:

① Address = URL

② Type of services = Method

③ Label on box = Headers

④ Inside the box = Body (data)

Example HTTP Request

POST / users HTTP/1.1

Host: example.com

Content-Type: application/json

{

"username": "shubham",

"age": 22

}

① Method: POST

② URL: / users

③ Header: Content-Type =

application/json

④ Body: JS on data



② What is a Response?

A response is what the server sends back.

It has:

- ① Status Code (200, 404)
- ② Headers
- ③ Body (data/message)

Example HTTP Response

HTTP/1.1 201 Created
Content-Type: application/json

```
{  
  "id": 10,  
  "name": "Shubham",  
  "age": 22  
}
```

③ Status Codes

Code	Meaning
200	OK (Success)
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
505	Server Error

④ What are Headers?

Headers = Metadata about the request/response

Content Headers:
Content-Type → What format is the data?



① Authorization → Token / login info

② Accept → what client wants back

Example:

Content-type: application/json

Authorization: Bearer <token>

⑤ What is body?

Body = Actual data you send or receive

Used mainly in:

① POST

② PUT

③ PATCH

Example body:

JSON

{ "email": "test@gmail.com"

"password": "1234"

}

⑥ See this in FAST API

from fastapi import FastAPI

app = FastAPI()

@app.post("/users")

def create_user(user: dict):

return

{ "message": "user created",

"user": user

}



if client send Post with
JSON body:

```
{  
  "name": "Shubham",  
  "age": 22.  
}
```

Response:

```
{  
  "message": "User created",  
  "user": {  
    "name": "Shubham",  
    "age": 22  
  }  
}
```

FastAPI:

- ⊙ Read body
- ⊙ Parse JSON
- ⊙ Send JSON response

⑦ Full Request → Response Flow

1. Client Send request
2. Server receive it
3. Server validate data
4. Server runs logic
5. Server send response
(status + header + body)

Q. Explain Process of
Parsing JSON.

JSON = JavaScript Object Notation
is a text based data interchange
format used to send data
between different systems,



Such as web server and a browser.

① Before Parsing (Serialization / Stringifying)

When data is sent over a network or stored in a file, it is first converted into a flat, structured string of text, which is called serialization.

② During Parsing (Deserialization)

When an application receives this string, it cannot work with Data structure. Parsing process involves:

a) Lexical Analysis - The Parser breaks the stream of character into meaningful unit called tokens.

b) Syntactic Analysis - It organizes these token into a hierarchical structure based on the strict rules of JSON grammar and often creating a Parse tree.

c) Conversion: Final structured data is converted into a native Data structure the programming language understands making individual data field and value accessible.